

Entrega 2 – Álgebra Linear, Vetores e Geometria Analítica

Aplicação de Transformações Lineares em Imagens Digitais
Projeto Interdisciplinar – Lideranças Empáticas (LE) – Scanner AI | 5CCOMP – 2026

Curso	Ciência da Computação
Turma	5CCOMP – 5º Semestre 2026
Projeto	Lideranças Empáticas (LE) – Scanner AI
Disciplina	Álgebra Linear, Vetores e Geometria Analítica
Entregável	Entrega 2 – Transformações Lineares em Imagens Digitais
Integrantes	Antonio Petri Leonardo Santos Lucas Gutierrez Vitor Kanashiro

Objetivo

Este documento apresenta a **Entrega 2** da disciplina de Álgebra Linear, Vetores e Geometria Analítica. O objetivo é demonstrar, de forma prática e conceitual, como **transformações lineares** atuam sobre dados matriciais — especificamente sobre a representação matricial da imagem do alimento gerada na Entrega 1. Partindo do arquivo *matriz_imagem_alimento.csv*, são aplicadas cinco transformações com matrizes explícitas 3×3, evidenciando o papel central da Álgebra Linear na manipulação computacional de imagens e em sistemas de Visão Computacional com IA.

Continuidade com a Entrega 1

Na Entrega 1, a imagem *Arroz.jpg* foi carregada com OpenCV, convertida para RGB e exportada pixel a pixel para o arquivo *matriz_imagem_alimento.csv* com as colunas *Linha_Y*, *Coluna_X*, *Red*, *Green*, *Blue*. A reconstrução da imagem a partir desse CSV comprovou que a informação visual está integralmente codificada na estrutura matricial.

Nesta Entrega 2, o ponto de partida é exatamente esse CSV: a imagem é reconstruída em memória lendo o arquivo gerado na etapa anterior, e as transformações lineares são aplicadas sobre essa matriz.

Carregamento da Imagem a partir do CSV

Leitura e reconstrução da matriz

O trecho abaixo substitui o carregamento direto do JPG. A imagem é inteiramente reconstruída a partir do CSV gerado na Entrega 1:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

df = pd.read_csv('matriz_imagem_alimento.csv')

altura = int(df['Linha_Y'].max()) + 1
largura = int(df['Coluna_X'].max()) + 1

img_rgb = np.zeros((altura, largura, 3), dtype=np.uint8)
img_rgb[df['Linha_Y'].values,
        df['Coluna_X'].values] = df[['Red', 'Green', 'Blue']].values
```

`h, w = altura, largura`

Coordenadas Homogêneas

Para representar transformações lineares e translações de forma unificada, cada ponto (x, y) é representado como vetor $[x, y, 1]^T$ em $\mathbb{R}^3$. Assim, qualquer combinação de rotação, escalonamento, cisalhamento e translação é expressa por uma única multiplicação de matriz 3×3 :

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Matrizes de Transformação

A tabela abaixo apresenta as cinco matrizes com seus parâmetros e determinantes:

Transformação	Matriz (3×3)	Parâmetros	det(M)
Escalonamento	$\begin{bmatrix} 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$S_x = 0.5$ $S_y = 0.5$	0.25
Rotação (45°)	$\begin{bmatrix} \cos \theta & -\sin \theta & t_x \\ \sin \theta & \cos \theta & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$\theta = 45^\circ$ (centralizada)	1.00
Cisalhamento	$\begin{bmatrix} 1 & 0.5 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$sh_x = 0.5$	1.00
Reflexão Horiz.	$\begin{bmatrix} -1 & 0 & w-1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	Eixo vertical central	-1.00
Projeção (Colapso Dim.)	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	Projeta $y \rightarrow 0$	0.00

Implementação em Python

Definição das matrizes

```
M_escala = np.array([[0.5, 0, 0],
                     [0, 0.5, 0],
                     [0, 0, 1]], dtype=np.float64)

theta = np.radians(45)
c, s = np.cos(theta), np.sin(theta)
diag = int(np.ceil(np.sqrt(w**2 + h**2)))
cx, cy = w / 2, h / 2
M_rotacao = np.array([
    [c, -s, -cx*c + cy*s + diag/2],
    [s, c, -cx*s - cy*c + diag/2],
    [0, 0, 1]], dtype=np.float64)

M_cisalhamento = np.array([[1, 0.5, 0],
                           [0, 1, 0],
```

```

[0, 0, 1]], dtype=np.float64)

M_reflexao = np.array([[-1, 0, w-1],
                        [ 0, 1, 0 ],
                        [ 0, 0, 1 ]], dtype=np.float64)

M_projecao = np.array([[1, 0, 0],
                        [0, 0, 0],
                        [0, 0, 1]], dtype=np.float64)

```

Mapeamento Inverso (Backward Mapping)

Para evitar buracos na imagem transformada, usa-se o **mapeamento inverso**: para cada pixel (x' , y') da imagem destino, calcula-se a posição de origem aplicando M^{-1} . Quando $\det(M) = 0$, a matriz é singular — aplica-se o mapeamento direto, resultando no colapso dimensional.

```

def aplicar_transformacao(img, M, novo_w=None, novo_h=None):
    src_h, src_w = img.shape[:2]
    dst_h = novo_h if novo_h else src_h
    dst_w = novo_w if novo_w else src_w
    img_out = np.zeros((dst_h, dst_w, 3), dtype=np.uint8)

    if abs(np.linalg.det(M)) < 1e-10:
        yy, xx = np.meshgrid(np.arange(src_h), np.arange(src_w), indexing='ij')
        coords = np.stack([xx.ravel(), yy.ravel(), np.ones(src_h * src_w)])
        dst = M @ coords
        xd = np.round(dst[0]).astype(int)
        yd = np.round(dst[1]).astype(int)
        mask = (xd >= 0) & (xd < dst_w) & (yd >= 0) & (yd < dst_h)
        img_out[yd[mask], xd[mask]] = img[yy.ravel()[mask], xx.ravel()[mask]]
    else:
        M_inv = np.linalg.inv(M)
        yy, xx = np.meshgrid(np.arange(dst_h), np.arange(dst_w), indexing='ij')
        coords = np.stack([xx.ravel(), yy.ravel(), np.ones(dst_h * dst_w)])
        src = M_inv @ coords
        xs = np.round(src[0]).astype(int)
        ys = np.round(src[1]).astype(int)
        mask = (xs >= 0) & (xs < src_w) & (ys >= 0) & (ys < src_h)
        img_out[yy.ravel()[mask], xx.ravel()[mask]] = img[ys[mask], xs[mask]]

    return img_out

```

Aplicando as transformações

```

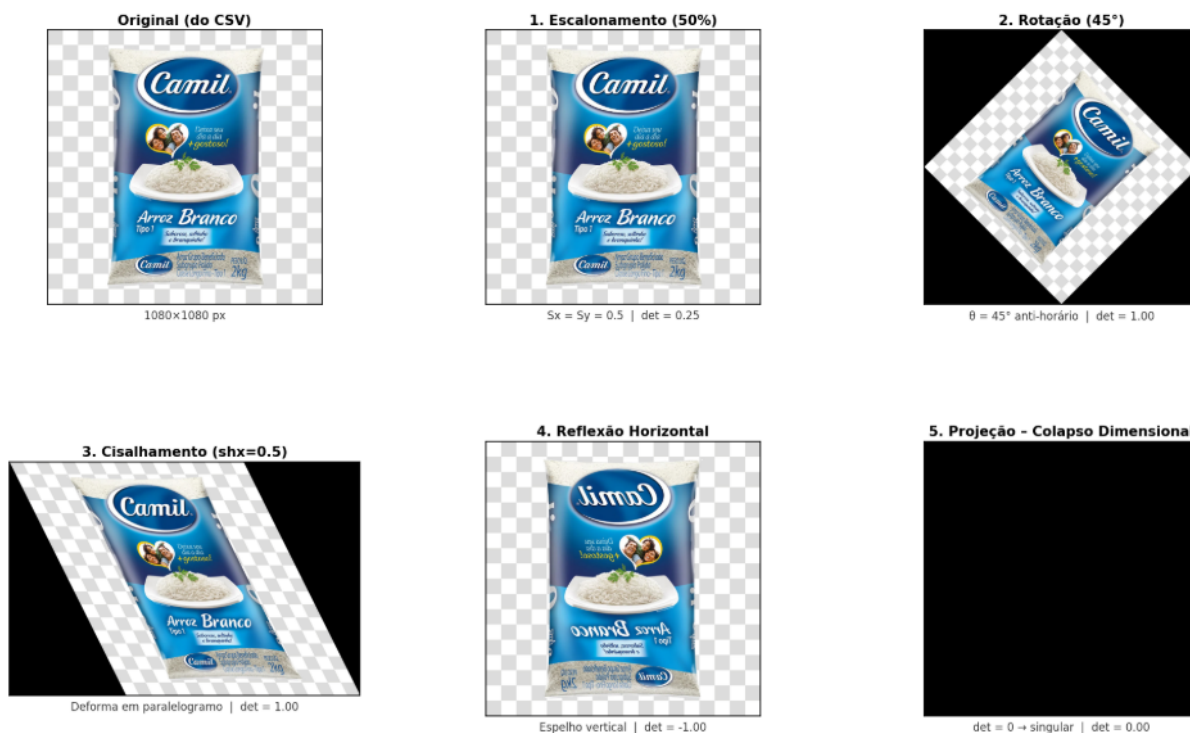
img_escala      = aplicar_transformacao(img_rgb, M_escala, novo_w=w//2, novo_h=h//2)
img_rotacao     = aplicar_transformacao(img_rgb, M_rotacao, novo_w=diag, novo_h=diag)
img_cisalhamento = aplicar_transformacao(img_rgb, M_cisalhamento, novo_w=w+int(0.5*h),
novo_h=h)
img_reflexao    = aplicar_transformacao(img_rgb, M_reflexao)
img_projecao    = aplicar_transformacao(img_rgb, M_projecao, novo_w=w, novo_h=h)

```

Resultados Visuais

A figura abaixo apresenta a imagem reconstruída do CSV (original) e as cinco transformações aplicadas, com os determinantes das matrizes em cada painel:

Transformações Lineares em Imagens Digitais - Entrega 2 Fonte: matriz_imagem_alimento.csv (Entrega 1)



Análise Conceitual das Transformações

Transformação	$\det(M)$	Efeito Geométrico	Propriedades Algébricas
Escalonamento ($S_x=S_y=0.5$)	0.25	Reduz a imagem a 50% em cada eixo. Área resultante = 25% da original.	Autovalores: S_x e S_y . Preserva paralelismo e proporções internas.
Rotação ($\theta = 45^\circ$)	1.00	Gira 45° anti-horário. Não altera forma nem tamanho.	M ortogonal: $M^{-1} = M^T$. Autovalores complexos: $e^{i\theta}$ e $e^{-i\theta}$.
Cisalhamento ($sh_x = 0.5$)	1.00	Converte retângulo em paralelogramo. Preserva área mas altera ângulos.	Autovalor duplo = 1. Apenas 1 autovetor real no eixo x.
Reflexão Horizontal	-1.00	Espelha em torno do eixo vertical. Inverte orientação (handedness).	Isometria: preserva distâncias. Autovalores: +1 e -1.
Projeção (Colapso Dim.)	0.00	Projeta todos os pixels em $y=0$. Perda irreversível de informação.	Matriz singular: sem inversa. $\dim(\ker)=1$: toda direção y vai a zero.

Colapso Dimensional

A transformação de projeção ilustra um conceito central da Álgebra Linear: quando $\det(M) = 0$, a matriz é **singular** e não possui inversa. O núcleo (kernel) tem dimensão não nula — toda a direção y é mapeada para zero. Informações distintas tornam-se indistinguíveis, tornando a recuperação da imagem original matematicamente impossível.

Na prática, esse fenômeno ocorre em Visão Computacional quando câmeras capturam cenas com perspectiva extrema ou oclusão total — situações onde parte da informação visual é irrecuperável.

Conclusão

Este trabalho demonstrou que **transformações lineares** são mecanismos fundamentais da computação gráfica e do processamento de imagens com IA. Partindo diretamente da representação matricial gerada na Entrega 1 (*matriz_imagem_alimento.csv*), ficou evidente que a estrutura matricial é suficiente para todas as operações de transformação. O determinante da matriz revela propriedades essenciais: com $\det \neq 0$ a transformação é invertível; com $\det = 0$ ocorre colapso dimensional e perda irreversível de informação. No projeto **Lideranças Empáticas – Scanner AI**, esses fundamentos embasam o pré-processamento do pipeline de Visão Computacional para classificação automatizada dos alimentos arrecadados.